

# Implementation Plan (v2)

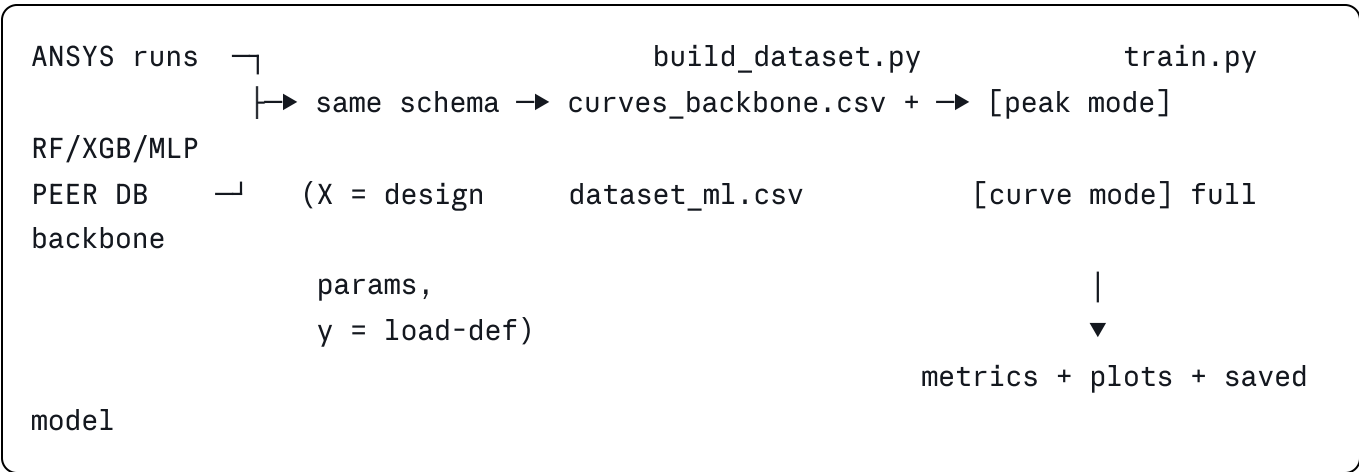
## Non-Linear Numerical Analysis of RCC Column with Machine Learning Prediction Model

**Revision note.** This version assumes the **training data for Phase 1 is the PEER-style rectangular RC column dataset**, and that your ANSYS simulations produce data in the **same schema** (column design parameters as features, load-deformation behaviour as the target). The earlier "material properties → stress-strain" framing is folded in as a special case:  $f'_c$  is the shared link, and  $\epsilon$  can be derived from  $f'_c$ .

**Goal.** Train an ML surrogate that predicts an RCC column's load-deformation response (peak capacity and/or the full backbone curve) from its design parameters, validated against experiment and reused for your ANSYS-driven study.

|              |                                                                                                                                                                           |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Features (X) | b, h, length/shear span L, $f'_c$ , longitudinal & transverse reinforcement ratios, bar/hoop details, axial load P and axial-load ratio, (+ displacement, for curve mode) |
| Targets (y)  | <code>peak_load_kN</code> and <code>disp_at_peak_mm</code> (scalar mode); full <code>lateral_load</code> vs <code>displacement</code> backbone (curve mode)               |
| Data sources | PEER UW rectangular DB (299 experimental specimens) + your ANSYS runs in the same columns                                                                                 |

### 0. Pipeline at a Glance



The two data sources share identical columns, so they can be **trained together** (more data) or kept separate (ANSYS validated against the experimental PEER set).

## Phase 1 — Assemble the training data (PEER + ANSYS, one schema)

*Deliverable: a single clean table per modelling mode.*

- ☐ Run `build_dataset.py` to download the 299 PEER rectangular specimens and produce `curves_backbone.csv`, `specimen_summary.csv`, `peak_loads_displacements.csv`.
- ☐ Export the PEER properties (`.wk1` → `properties_rect.csv`) and merge → `dataset_ml.csv` (features beside targets).
- ☐ **Export your ANSYS results into the same columns** (`specimen_id, specimen, b_mm, h_mm, L_mm, fc_MPa, rho_long, rho_trans, P_axial_kN, axial_load_ratio, displacement_mm, lateral_load_kN`). Tag each row with a `source` column (`peer` / `ansys`) so you can train on either or both.
- ☐ Concatenate PEER + ANSYS into `dataset_ml.csv` (peak) and `curves_backbone.csv` (curve).

**Outcome:** one consistent dataset where a row is a specimen (peak mode) or a curve point (curve mode), regardless of whether it came from test or ANSYS.

---

## Phase 2 — Data audit

*Deliverable: a 1-page sanity note.*

- ☐ **Variation check:** histogram every feature — confirm  $f'_c$ , geometry, reinforcement and axial load actually vary across your rows.
  - ☐ **Collinearity check:** correlation matrix. Watch for the geometry/axial-load couplings (e.g.  $\text{axial\_load\_ratio} = P/(f'_c \cdot b \cdot h)$  is derived from other columns). Drop or keep derived features deliberately.
  - ☐ **Missing values:** the PEER properties have gaps — `train.py` median-imputes, but note which columns are sparse.
  - ☐ **Units:** keep everything SI (mm, kN, MPa). ANSYS exports must match.
  - ☐ **Distribution match (PEER vs ANSYS):** overlay feature histograms by `source` to confirm your ANSYS runs cover the same space as the experiments.
- 

## Phase 3 — Pick the modelling mode

*Deliverable: chosen target + leakage-free split.*

|          | Peak mode (scalar)                           | Curve mode (backbone)                                                    |
|----------|----------------------------------------------|--------------------------------------------------------------------------|
| Row      | one specimen                                 | one curve point                                                          |
| X → y    | design params →<br><code>peak_load_kN</code> | design params + <code>displacement</code> →<br><code>lateral_load</code> |
| File     | <code>dataset_ml.csv</code>                  | <code>curves_backbone.csv</code> + properties                            |
| Use when | you need capacity/ductility                  | you need the whole load-deformation curve                                |

- ☐ Start with **peak mode** (fastest signal), then move to **curve mode** for the full response (this is the analogue of your ANSYS stress-strain curve).
- ☐ Always split by `specimen_id` (built into `train.py` via `GroupShuffleSplit`).

## Phase 4 — Baselines & model comparison

*Deliverable: a metrics leaderboard.*

- ☐ Run `train.py` (peak mode) — it trains **Random Forest**, **XGBoost**, **MLP** and reports **R<sup>2</sup>**, **RMSE**, **MAE** plus grouped-CV R<sup>2</sup>.
- ☐ Read `metrics_peak.csv` and `parity_peak.png`. Note the best model.

```
bash

pip install -r requirements.txt
python train.py --mode peak --data ./out/dataset_ml.csv --target peak_load_kN
```

## Phase 5 — Tune the leading models

*Deliverable: tuned models + records.*

- ☐ Tune **XGBoost** (`(n_estimators, max_depth, learning_rate, subsample)`) and the **MLP** (`(hidden_layer_sizes, alpha, learning_rate)`). Add **Optuna** for search.
- ☐ Keep **Random Forest** as the robust baseline.
- ☐ (Optional) **CatBoost** for native handling of categorical columns (test configuration, failure mode).

## Phase 6 — Full backbone-curve model

*Deliverable: predicted load-deformation curves.*

- ☐ Run `train.py` (curve mode): `displacement` becomes a feature, so sweeping it reconstructs the whole curve.
- ☐ Inspect `curves_pred_vs_actual.png` (dots = actual, lines = predicted).
- ☐ (Optional) For sequence output, follow the **LSTM** approach (Fan et al. 2023).

```
bash
```

```
python train.py --mode curve --curves ./out/curves_backbone.csv --properties ./
```

---

## Phase 7 — Evaluation & ANSYS validation

*Deliverable: the core results section.*

- ☐ Report **R<sup>2</sup>**, **RMSE**, **MAE** + curve-level errors (peak-load %, peak-disp %).
- ☐ **Train on PEER, test on ANSYS** (and vice-versa) using the `source` tag — this directly checks whether your ANSYS model reproduces experimental behaviour.
- ☐ Benchmark against a design-code capacity equation (ACI/Eurocode) to show ML adds value.
- ☐ **SHAP** feature importance to confirm physically sensible drivers ( $f'_c$ , section size, axial load).

---

## Phase 8 — Package & write up

- ☐ Save best models (`best_model_peak.joblib`, `best_model_curve.joblib`).
  - ☐ Wrap a `predict()` helper / small notebook for new columns.
  - ☐ Write: data assembly, audit, model comparison, ANSYS-vs-PEER validation, SHAP, limitations. Cite each specimen's original reference + the PEER manual.
-

## Files in this package

| File                                 | Purpose                                                             |
|--------------------------------------|---------------------------------------------------------------------|
| <code>build_dataset.py</code>        | download + assemble the PEER data into ML-ready CSVs                |
| <code>train.py</code>                | train/compare RF · XGBoost · MLP (peak & curve modes)               |
| <code>requirements.txt</code>        | install everything ( <code>pip install -r requirements.txt</code> ) |
| <code>properties_template.csv</code> | schema for the features file                                        |
| <code>specimen_index_full.csv</code> | all 301 specimens (id, author, year, curve URL)                     |

## How to run, start to finish (VS Code)

```
bash

# 1. environment
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\
pip install -r requirements.txt

# 2. build the dataset (downloads 299 curves)
python build_dataset.py --out ./out                    # add --properties prop

# 3. train the scalar peak-load model
python train.py --mode peak --data ./out/dataset_ml.csv --target peak_load_kN

# 4. train the full backbone-curve model
python train.py --mode curve --curves ./out/curves_backbone.csv --properties ./
```

Outputs land in `./train_out/` (metrics CSVs, parity/curve PNGs, saved models).

## Risk register

| Risk                       | Trigger                                           | Action                                              |
|----------------------------|---------------------------------------------------|-----------------------------------------------------|
| ANSYS ≠ PEER feature space | Phase 2 histograms by <code>source</code> diverge | widen the ANSYS DOE to cover the experimental range |

| Risk                                 | Trigger                                    | Action                                              |
|--------------------------------------|--------------------------------------------|-----------------------------------------------------|
| Collinear / derived features         | high correlations                          | drop or keep deliberately; document                 |
| Sparse properties                    | many NaNs in a column                      | impute (built in) or drop the column                |
| Data leakage                         | curve points split across train/test       | always split by <code>specimen_id</code> (built in) |
| Cyclic vs monotonic confusion        | training on hysteresis instead of backbone | use <code>curves_backbone.csv</code> for curve mode |
| Overfitting (~hundreds of specimens) | train $R^2 \gg$ test $R^2$                 | regularise, simpler model, combine PEER+ANSYS       |

## Indicative timeline

| Week | Focus                                    |
|------|------------------------------------------|
| 1    | Phase 1–2: assemble PEER+ANSYS, audit    |
| 2    | Phase 3–4: split + baselines (peak mode) |
| 3    | Phase 5: tune XGBoost / MLP              |
| 4    | Phase 6: backbone-curve model            |
| 5    | Phase 7: evaluation, ANSYS-vs-PEER, SHAP |
| 6    | Phase 8: package + write up              |